Application Audit Report — Mint (iPreneur) AI Pitch Deck Generator

Functional, Architectural, Security, Compliance and Operational Review

Prepared for the Company Audit — Infopace

3 November 2026

Contents

Document Control
- How to read this report
- Scope, method and evidence limits

1. Executive Summary
- 1.1 What the application is
- 1.2 Overall assessment
- 1.3 Key strengths
- 1.4 Principal findings requiring management attention
- 1.5 Priority recommendations (30-day view)

2. Application Overview
- 2.1 Purpose and value proposition
- 2.2 Users and roles
- 2.3 Product maturity
- 2.4 Technology inventory
- 2.5 Size and composition

3. Functional Description — Features
- 3.1 Public landing page (`/`)
- 3.2 Authentication and account
- 3.3 Dashboard (`/dashboard`)
- 3.4 New-deck intake (`/projects/new` , 1,344-line form)
- 3.5 Deck types
- 3.6 Company profile — the master parameter set
- 3.7 AI generation pipeline
- 3.8 Claims auditor (`agents/audit/claims_auditor.py`)
- 3.9 Rendering, themes, editing and export
- 3.10 Billing (`/billing`)

4. Architecture
- 4.1 Logical architecture
- 4.2 Backend module map
- 4.3 Data model
- 4.4 Concurrency model
- 4.5 Configuration
- 4.6 Deployment topology — as documented vs. as evidenced

5. Interfaces
- 5.1 External interfaces
- 5.2 API summary

6. Security Assessment
- 6.1 Control summary
- 6.2 Findings
- 6.3 Positive security observations

7. Data Governance, Privacy and Compliance
- 7.1 Data inventory

7.2 Findings
7.3 Regulatory mapping (indicative, not legal advice)
8. AI-Specific Risk Assessment
9. Billing and Financial Controls
10. Quality Assurance
10.1 Automated tests — executed for this audit
10.2 Findings
10.3 Defects recorded and corrected (from project log) — evidence of an active fix loop
11. Operations, Resilience and Maintainability
11.1 Positive operational observations
12. Risk Register and Remediation Roadmap
12.1 Risk register
12.2 Prioritised roadmap
Appendix A — Repository Inventory
Appendix B — Deck-Type Catalogue
Appendix C — Parameter Groups (924 parameters in 49 groups)
Appendix D — Theme Library (40)
Appendix E — API Endpoint Inventory
Appendix F — Environment Variable Catalogue
Appendix G — Test Execution Record
Appendix H — Evidence Index (selected)
Appendix I — Glossary

Document Control

Item	Detail
Application under audit	**Mint** (internal codename **iPreneur**) — AI pitch-deck generator
Repository	`github.com/Infopace/Pitch-Deck` (monorepo; application in `ipreneur/`)
Snapshot audited	Branch `claude/charming-goodall-kbzp5f`, HEAD `2c3a010` ("perf(export): fix the real cause of slow PPTX export, add progress toast")
Repository activity	57 commits, single committer, project log entries from 20 May 2026 to 27 Oct 2026, last commit 30 Oct 2026
Report date	3 November 2026
Owner named in repository	Infopace Management Pvt. Ltd. (README licence line: "Proprietary — © Infopace Management Pvt. Ltd."; `ipreneur/README.md` instead says "iPreneur Inc." — see finding D-02)
Classification	Confidential — for audit use
Method	Static source review of every tracked source file in scope, configuration and deployment-file review, repository-history review, and execution of the offline automated test suite

How to read this report

- **Section 1** is a two-page executive summary for readers who need only conclusions.
- **Sections 2–5** describe what the application is, what it does, how it is built, and what its interfaces are.
- **Sections 6–11** are the assessment: security, data governance and compliance, AI-specific risk, billing controls, quality assurance, and operations. Every finding carries an identifier (`S-`, `D-`, `A-`, `B-`, `Q-`, `O-`), a severity, the evidence (file and line) and a recommendation.
- **Section 12** consolidates the risk register and a prioritised remediation roadmap.
- **Appendices** hold inventories: deck types, parameter groups, themes, environment variables, endpoints, test results and a glossary.

Scope, method and evidence limits

In scope. All tracked files under `ipreneur/` (backend, frontend, infrastructure, scripts, shared types, docs), the root-level deployment and CI files (`render.yaml`, `.github/workflows/`), the project log, and committed research and design assets.

Method.

1. Full read of the backend application (`ipreneur/backend/app`, 12,842 lines of Python across 70+ modules) including every API endpoint, the ORM models, the database layer, AI agents, the crawler, the renderer, the profile engine and the storage layer.
2. Read of the frontend routing, state, API client, authentication store, billing integration, deck-export code and the principal pages (13,471 lines of TypeScript/TSX plus roughly 2,600 lines of CSS).
3. Review of deployment artefacts (Dockerfiles, Compose file, Render blueprint, Firebase/Cloudflare/Vercel configuration, GitHub Actions workflow).
4. Pattern scan of tracked files for credentials, and review of repository content for data that should not be under version control.

5. Execution of the automated tests that can run offline (results in Section 10 and Appendix G).
6. Review of the 1,374-line project log (`ipreneur/PROJECT_LOG.md`) for decisions, incidents and declared-but-unremediated items.

Evidence limits — please read. This is a *source-code and repository* audit. The auditor had **no access** to the live production environment, the cloud consoles (Google Cloud, Firebase, Supabase, Render, Cloudflare, Razorpay), the production database, runtime logs, Sentry, or the people who operate the system. Consequently:

- Statements about the deployed environment (for example, which host serves the API, or whether a secret has been rotated) are taken from repository files and the project log and are labelled **“per repository”** or **“per project log”**. They are not independently verified.
- Findings flagged **“verify in deployment”** depend on runtime configuration that cannot be seen from source.
- Nothing in this report was obtained by attacking, scanning or probing any running system. No exploit was executed.

Assumptions stated in line with the brief.

- “The application under audit” is taken to be the Mint/iPreneur platform in this repository.
- Where the repository’s own documents disagree (README vs. architecture document vs. code vs. deployment files), **the code is treated as authoritative** and the disagreement is recorded as a documentation finding.
- Severity uses a conventional four-level scale — **High**, **Medium**, **Low**, **Informational** — assessed on likelihood and impact *for this application’s actual deployment shape* (a small SaaS holding founders’ confidential business data and processing paid subscriptions). No finding is rated “Critical” because none, on the evidence available, gives an unauthenticated party direct access to stored customer data; the highest-rated items concern cost abuse, server-side request forgery, and exposed credentials.
- This report is not legal advice. Regulatory observations (Section 7) identify where the design touches a regime; counsel should confirm applicability.

---

1. Executive Summary

1.1 What the application is

Mint is a web-based Software-as-a-Service product that turns a company website (or a short founder questionnaire, when there is no website) into a complete, investor-grade presentation. A user pastes a URL or completes an intake form, chooses one of **14 deck types**, and the system crawls the website, researches the market and competitors with a web-grounded large language model (Google Gemini), writes every slide, validates and audits the result, and renders it in **40 visual themes**. The user can then edit text in place, switch themes, and download a PowerPoint (`.pptx`) file. A free plan and a paid Pro plan (₹2,500/month, via Razorpay) are offered, and the product is explicitly localised for Indian founders (rupee amounts, lakh/crore notation).

1.2 Overall assessment

Dimension	Rating	One-line rationale
Functional completeness	**Strong**	End-to-end flow works; 14 deck types, 924-parameter profile model, 40 themes, in-place editor, two export paths, billing.
Domain engineering quality	**Strong**	Single-source-of-truth deck registry; schema-validated AI output; deterministic claims auditor; provenance-tracked facts; currency integrity.
Application security	**Needs improvement**	Good fundamentals (bcrypt, JWT expiry, tenant-scoped queries, signed payments, production boot guard) but one unauthenticated cost-bearing endpoint, no SSRF defence, committed API keys, and unenforced plan limits.
Data governance & privacy	**Needs improvement**	No hard-delete, retention policy or account erasure; third-party AI processing of confidential data; customer-style artefacts committed to version control.
Compliance posture	**Not evidenced**	No certification, policy set, audit log, consent capture or DPA is present in the repository.
Quality assurance	**Moderate**	559 backend tests pass (domain logic well covered); no CI test gate, no frontend tests, no API-security tests.
Operability & resilience	**Moderate**	Sensible guards (orphaned-job reconciliation, schema-drift check, Sentry hook) but generation runs in in-process threads, deployment documentation has drifted, and migrations are manual.
Documentation	**Needs improvement**	The project log is excellent; the README and architecture document describe an earlier design (Claude, Stripe, WebSockets, refresh tokens).

1.3 Key strengths

1. **A well-factored domain core.** A single deck registry (`app/decks/registry.py`) declares each deck type once; the prompt, validators, renderer and frontend all derive from it, and a test (`test_deck_registry_frontend_sync.py`) enforces that the Python and TypeScript copies cannot drift.
2. **AI output is treated as untrusted.** Every generated section is validated independently against a typed schema with length caps; a bad section is dropped rather than breaking the deck. A deterministic *claims auditor* then checks the result for invented revenue, market funnels that point the wrong way, percentages that do not total 100, unevidenced certifications and leftover placeholder text.

3. **Provenance-aware data model.** Every company fact carries *source, method, confidence and as-of date*, with an explicit trust ranking (founder > derived > crawl > research). This is the foundation for an evidence appendix and is uncommon in products of this type. (Today it is an engine and API capability; the user-facing profile screen has not yet been built — see Section 3.6.)
4. **Defensive production start-up.** In production the service refuses to boot with placeholder secrets, a short JWT secret, debug on, missing AI key, or `localhost` in CORS.
5. **Payment integrity.** Razorpay checkout results and webhooks are verified by HMAC-SHA256 with constant-time comparison; administrative roles cannot be downgraded by a payment event.
6. **Honest engineering record.** The project log records incidents, root causes and *unremediated* items candidly (for example, exposed credentials in history), which materially aids an audit.

1.4 Principal findings requiring management attention

ID	Severity	Finding
S-01	**High**	`POST /api/v1/pipeline/generate` is **unauthenticated and unthrottled** and launches a headless-browser crawl plus three paid LLM calls — a direct cost-abuse and denial-of-service vector.
S-02	**High**	**No server-side-request-forgery (SSRF) protection.** Any submitted URL — including `localhost`, private ranges and cloud metadata addresses — is fetched by a headless browser; image fetches follow redirects to any host.
S-03	**High**	**Credentials in version control.** Eight occurrences of two Google-API-key-format strings are hard-coded in tracked scripts; the project log additionally records real database/storage credentials committed earlier and **not rotated**.
B-01	**High**	**Plan limits are advertised but not enforced.** The Free plan promises 3 decks/month; no server code checks or increments usage. Only per-IP rate limits exist. Direct revenue and AI-cost exposure.
S-04	**Medium**	A project can be attached to **another user's company profile** by ID (missing ownership check) — an insecure direct object reference mitigated only by UUID unguessability.
S-05	**Medium**	Session token stored in browser `localStorage`, with no refresh, no revocation and no security headers (CSP) on the single-page app.
S-06	**Medium**	Uploads trust the client's content-type, permit SVG, and are served unauthenticated from local ephemeral disk.
D-01	**Medium**	**No data-subject lifecycle:** soft-delete only, no purge, no account deletion/export, no retention schedule; generated files and uploads survive project deletion.
D-03	**Medium**	**55 generated decks, 21 uploaded images and scraped-data JSON files are committed to the repository**, plus ~7 binary design archives and install logs.
O-01	**Medium**	Deck generation runs in **in-process daemon threads**, not a queue; it is lost on restart, unbounded in concurrency, and cannot scale horizontally. The Celery stack in the repo is legacy and unwired.
Q-01	**Medium**	**No CI test or security gate**; the only workflow deploys the frontend. No frontend tests exist.

1.5 Priority recommendations (30-day view)

1. Put authentication and rate limiting on, or delete, `/pipeline/generate`. Add destination validation (block loopback, private, link-local, and metadata ranges; re-validate after redirects) to the crawler and image fetcher.
2. **Rotate** the exposed Google API keys and the credentials recorded in project-log entry #30; remove the key-bearing scripts; scrub history.

3. Enforce plan quotas server-side (check at create/analyze; increment `projects_used_this_month` / `exports_used_this_month`; monthly reset).
4. Add an ownership check when attaching `profile_id`; move session tokens to `HttpOnly` cookies or add CSP and a revocation/refresh design.
5. Add a CI workflow that runs the offline backend tests, a type-check/build of the frontend, dependency and secret scanning; pin backend dependencies.
6. Define and implement a data-lifecycle policy (hard-delete, account erasure, retention) and a user-facing privacy notice covering AI sub-processors.

---

2. Application Overview

2.1 Purpose and value proposition

Founders need decks for several different rooms — investors, customers, partners, boards, demo days — and most tools only produce generic slides. Mint's proposition (taken from the product's own landing page and README) is: *"Transform any company URL into an investor-grade pitch deck in minutes,"* with each of 14 deck types having its own slide sequence and writing rules, claims checked against the data the founder supplied, and Indian-market conventions built in.

2.2 Users and roles

Role (`users.role`)	How obtained	Limits (per `PLAN_LIMITS`)	Notes
`free`	Default at registration	3 projects, 3 exports	Limits are *reported* by the API but **not enforced** (finding B-01).
`pro`	Razorpay subscription verified or webhook-confirmed	50 projects, 50 exports	₹2,500/month (display text; actual charge is defined by the Razorpay plan).
`enterprise`	Set manually	Unlimited	"Custom branding, API access, team collaboration" are *marketed* (`/billing/plans`) but not implemented in code.
`admin`	Set manually	Unlimited	No administrative endpoints or console exist in the code.

Payment events can only move accounts between `free` and `pro` ; `enterprise` / `admin` are protected.

2.3 Product maturity

The repository's own phase list is out of date. From the code and project log the actual state is:

Capability	State
Registration, login, session	Implemented (no refresh, verification or reset)
Website crawl + AI research + deck generation	Implemented, in production per log
14 deck types, intake forms driven by the registry	Implemented
Company profile (924 parameters, provenance)	Engine, API and tests implemented. **Backend-only today:** the frontend has no profile screen and no profile API client (`src/services/api/` contains only `billing` , `client` , `deckTypes` , `projects`), and the New Deck form never sends `profile_id` . Profile UI and AI pre-fill are listed as paused in the project log (#52).
Claims auditor	Implemented and stored per deck; **no UI displays the findings**

Capability	State
Theme library (40) and in-place text editor	Implemented
PPTX export (client-side images and server-side editable)	Implemented (two paths — see 3.9)
PDF export	Advertised in plan text; **not implemented** (the `format` parameter is accepted but only PPTX is produced)
Razorpay billing	Implemented; per log, production keys/plan were not yet configured at 27 Oct 2026
Celery background workers	Present but legacy/unwired
Team collaboration, API access, custom branding (Enterprise)	Marketed; not implemented

2.4 Technology inventory

Layer	Technology (per code and manifests)
Frontend	React 18.3, TypeScript 5.4 (strict), Vite 5, Tailwind CSS 3.4, React Router 6, TanStack Query 5, Zustand 4, React Hook Form + Zod, Radix UI primitives, Recharts, `motion`, `html-to-image`, `pptxgenjs`, `react-hot-toast`, `lucide-react`
Backend	Python 3.11 (container), FastAPI, Uvicorn, SQLAlchemy 2 (async), asyncpg / aiosqlite, Pydantic v2 + pydantic-settings, `slowapi` (rate limiting), `python-jose` (JWT), `bcrypt`, Loguru, `httpx`, `boto3`, Sentry SDK
AI	Google Gemini via direct REST (`generativelanguage.googleapis.com`), model `gemini-3.8-flash` per configuration, Google Search grounding on the research pass, two API keys to split quota
Crawling	Playwright 1.52 (headless Chromium), BeautifulSoup4, lxml
Document generation	`python-pptx` (server, editable output); `pptxgenjs` + `html-to-image` (browser, image output)
Data	PostgreSQL 15 (Supabase in production per log; Compose uses `postgres:15-alpine`), JSON columns for AI payloads; SQLite fallback for development
Cache / queue	Redis 7 and Celery (declared in Compose; **not on the live generation path**)
Object storage	S3-compatible (Supabase Storage in production per log; MinIO in Compose) with local-disk fallback
Payments	Razorpay subscriptions
Hosting (per repository/log)	Frontend: Firebase Hosting, project `pitch-deck-e1d6c`, custom domain `pitchdeck.infopaceindia.com`. Backend: Google Cloud Run, region `asia-northeast1`, auto-deployed by a Cloud Build trigger. Also present and not authoritative: `render.yaml` (Render), `wrangler.jsonc` (Cloudflare), `vercel.json` (Vercel).
CI/CD	One GitHub Actions workflow (frontend deploy) using Workload Identity Federation; backend deploy via external Cloud Build trigger

2.5 Size and composition

Measure	Value
Tracked files	361 (of which 76 are generated/uploaded customer-style artefacts — see D-03)
Backend application code	12,842 lines Python (`app/`)
Backend tests	1,656 lines across 11 test modules; 559 offline tests pass, 1 skipped
Frontend code	13,471 lines TS/TSX; 2 large CSS files (`index.css` 1,716 lines; hero CSS 859 lines)
Deck section specifications	65 (20 of which are evidence/appendix sections)
Deck types	14
Profile parameters	924 in 49 groups
Themes	40 (4 families)
API endpoints	30 versioned routes under `/api/v1` , plus `/health` , the `/uploads` static mount and (non-production) `/docs` (Appendix E)
Database tables	6 (`users` , `projects` , `company_profiles` , `presentations` , `jobs` , `subscriptions`)

3. Functional Description — Features

3.1 Public landing page (`/`)

- Hero with animated “story” graphic; brand mark (geometric M-cube logo).
- Live theme preview: selecting a theme swatch re-renders a *different sample deck variant* in that theme, using the same renderer as the product (`sampleDeckVariant`).
- “What’s actually behind the deck” trust section describing the two-pass research and the audit pass.
- Deck-type gallery grouped by audience, derived from the registry (so the count shown always matches the code).
- Template gallery (`/templates`) with pre-selection that carries into the new-deck flow.
- Branded offline fallback: a minimal service worker (`public/sw.js`) serves `offline.html` instead of the browser’s native error page when a navigation fails. It deliberately does **not** cache the app.
- Three-tier branded error pages (404 “retro TV”, crash screen, offline screen) with a React `ErrorBoundary` inside the router.

3.2 Authentication and account

Capability	Behaviour
Register	Email, name, password. Email uniqueness enforced (HTTP 409 on duplicate). Password hashed with bcrypt. Returns user and a signed JWT (HS256, 30-minute expiry).
Login	Email/password; identical “Invalid credentials” message for unknown user and wrong password.
Logout	Server endpoint returns success but does nothing; the client clears local state.
Session	Bearer token on every API call; the Axios interceptor clears auth on any 401.
Route protection	`AuthGuard` redirects unauthenticated users to `/login` .
Settings page	Read-only display of name, email and plan.
Not present	Password reset, email verification (the `is_verified` column exists but is never set), password-strength rules, MFA, session listing, account deletion, profile editing.

3.3 Dashboard (`/dashboard`)

Project grid with live cover thumbnails (rendered by the same slide renderer, wrapped in a `FitSlide` scaler), status filters, search/sort, an accelerator-style executive layout, and statistics that were deliberately restricted to real data (project log #49 records removal of fabricated metrics). Projects are listed 20 per page, newest-updated first, scoped to the signed-in user.

3.4 New-deck intake (`/projects/new` , 1,344-line form)

- **Deck-type picker** rendered from `GET /deck-types` with category tabs and a “slide running order” side panel that previews the exact sequence the chosen type will generate.
- **With website:** company URL plus only the fields the chosen deck type actually uses (the registry’s `intake_fields` drive which inputs appear; hidden fields are stripped from the payload so a value typed for a previously selected type never reaches the prompt).

- **Without website (idea-stage founders):** manual intake — company name, industry, problem, solution, target customer, traction/validation notes, known competitors, founders (name, role, one-liner, email, phone, LinkedIn).
- **Deck-specific briefs:** e.g. Sales asks for target buyer, buyer pain, citable case studies and pricing; Partnership asks for partner name, what the partner brings, and the ask; Internal decision memo asks for initiative, decision requested, options considered, resources and risks; Investor Update asks for period, prior metrics, wins and challenges.
- **Numbers and branding:** funding stage, funding round, business model, ARR/MRR, customers, MAU, burn, runway, growth, CAC/LTV, churn, NRR and the ask, with a **₹/$ toggle that defaults to INR**; every label spells out its abbreviation and gives a one-line definition.
- **Assets:** logo and up to five gallery photos (only shown for deck types that contain a gallery slide).
- **Presentation format:** presenter-led vs. standalone (controls on-slide text density).
- **Draft persistence:** the in-progress form is saved to the device so a refresh does not lose work.
- **Validation:** type-specific required fields are enforced on both client and server (`ProjectCreate` model validator).

3.5 Deck types

Fourteen deck types, each with its own audience, goal, writing rules, slide sequence, research depth and (where relevant) stage-specific variants. Full catalogue in Appendix B. Summary:

Group	Types
Raise capital	Investor / Fundraising (24 slides; 22 for idea/pre-seed with a Vision slide replacing Traction), Elevator (5), Demo Day (10), Business Plan (25), Extended/Master (25)
Win customers	Sales / Customer (14), Product / Demo (14), Go-to-Market (22)
Align partners & stakeholders	Partnership (15), Internal / Decision Memo (10), Investor Update (8)
Explain the company	General / Company Overview (20), Vision / Mission (8)
Evidence	Appendix / Evidence (2 fixed bookends + sections computed from the company profile)

Research depth is declared per type — **full** (deep web-grounded research), **light**, or **none** (the Internal memo and Investor Update rely entirely on founder-supplied facts, so nothing is researched or invented).

3.6 Company profile — the master parameter set

- **Status: backend capability, not yet user-facing.** The engine, API and tests exist and the generation pipeline consumes a profile when a project carries a `profile_id`, but no frontend screen creates or edits profiles and the New Deck form does not send `profile_id`. Today the profile can be exercised only through the API.
- A **per-company profile** (`company_profiles`) outlives any single deck; parameters are collected once and *selected from* by each deck type.
- **924 parameters in 49 groups** (Appendix C) covering identity, team, problem, customer, product, technology/IP, security & compliance, market, business model, traction, unit economics, go-to-market (with 12 sub-groups), sales, operations, partnerships, competition, moat, financials, fundraising, cap table, strategy, roadmap, risks, legal, evidence, and more.
- **Relevance matrix (R/C/O/A/N)** declares, per deck type and per group (with parameter-level overrides), whether a parameter is *Required, Conditional, Optional, Auto-derived,* or *Not applicable*. Stage rules are

encoded centrally — for pre-revenue companies, revenue parameters drop to Conditional and validation signals (pilots, LOIs, waitlist) rise to Required, so a founder is never asked to invent an ARR.

- **Provenance on every value:** `source` (founder / crawl / research / derived), `source_url`, `method`, `confidence` (0–1) and `as_of`. Source trust ranking: founder 1.0, derived 0.9, crawl 0.75, research 0.6. A crawl-sourced write cannot overwrite a founder answer (covered by test `test_crawl_sourced_write_cannot_overwrite_a_founder_answer`).
- **Derived values** (e.g. LTV:CAC) are computed on create and on every patch.
- **API:** schema (filtered by deck type and stage), list, create, read (with completeness by group), partial-merge patch (so two browser tabs cannot clobber each other), readiness check ("what is still missing for this deck type"), delete. Unknown parameter names are rejected (HTTP 422).

3.7 AI generation pipeline

Triggered by project creation with `start_analysis`, or by `POST /projects/{id}/analyze`. Progress is written to a `jobs` row and polled by the browser every few seconds.

Step	Progress	What happens
1. Crawl	0 → 25 %	Only if a website was given. Playwright BFS crawl: seeds from `sitemap.xml` and the homepage; priority-scored paths (team/leadership 95, about 90, product 85, pricing 80, customers 70 …); at most 5 pages per category; 4 pages in parallel; default cap 18 pages, 30-second page timeout; extracts title, description, headings, body text, images, CSS colours, JSON-LD, pricing signals and team mentions. Same-domain only; skips media, API, admin and static paths.
2a. Research pass	25 %	Skipped for "none" types. Gemini with Google Search grounding (key 2) produces founders, competitors, market data and funding history, with source URLs captured from the grounding metadata. Founder-supplied team members **override** any web-found "founders" (protects against same-name companies). A separate team-search call handles team discovery.
2b. Generation pass	25 → 75 %	Gemini (key 1) formats the researched facts into the deck's exact section sequence in one structured-JSON call. Up to four attempts on transient errors (429/5xx). **Truncation is detected** (finish-reason and token budget) and reported rather than parsed as a short deck. Lenient JSON repair for trailing commas and similar defects.
2c. Validation and audit	within 2b	Each section validated against its Pydantic model and length caps; invalid sections dropped with a logged reason. The deterministic claims auditor runs and its report is stored under `research_data.audit`.
3. Assets	75 %	Logo fetched (raster-normalised through Pillow); uploaded gallery images read from disk.
4. Render	80 %	`python-pptx` renders the PPTX using the user's chosen theme, or the theme the AI suggested.
5. Store and persist	95 → 100 %	PPTX uploaded to S3-compatible storage (local-disk fallback); project set to `ready`; `branding_data` merged (founder inputs preserved so a retry keeps them); `presentations` row created.

Failure handling: errors are recorded on the job and project; rate-limit errors are rewritten to a friendly "RATE_LIMITED … wait 1–2 minutes" message; on start-up the service marks any job left `running` / `queued` by a previous process as failed (`_reconcile_orphaned_jobs`).

3.8 Claims auditor (`agents/audit/claims_auditor.py`)

Deterministic checks, each producing an `error`, `warning` or `info` finding; the deck gets a 0–100 score (100 − 15 per error − 5 per warning). Checks:

1. Empty slides. 2. Placeholder text (`TBD` , `lorem ipsum` , “Competitor 1”, “[Company]” …). 3. Percentage splits (use of funds, cap table) not totalling 100. 4. Market funnel must narrow (TAM ≥ SAM ≥ SOM). 5. Revenue figures must match founder input in magnitude (±5 %) and currency; ARR/MRR/revenue appearing when none was supplied is flagged as invented. 6. Pre-revenue stage showing a growth curve. 7. Certifications (SOC 2, ISO 27001, HIPAA, GDPR, PCI-DSS, FedRAMP) claimed without evidence or a hedge (“in progress”, “planned”…). 8. Competition-table sanity (too few competitors; “wins every row”). 9. Roadmap items without a quarter label. 10. Investor red flags (“no competition”; growth percentage with no period). 11. Unit-economics benchmarks (LTV:CAC < 3:1; payback > 18 months).

Findings are advisory. They never block or alter a deck, and — importantly for management — **no user interface currently displays them** (finding Q-04), although the landing page advertises “the audit pass that checks it”.

3.9 Rendering, themes, editing and export

- **40 themes** in four families (Classic 10, Studio 10, Volume 2 10, Infographic 10); light and dark; fixed font names chosen for guaranteed availability (title fonts Segoe UI, Georgia, Trebuchet MS, Century Gothic, Consolas; body Calibri). Full list in Appendix D. Theme tokens exist in Python (`ppt/engine/themes.py`) and TypeScript (`themes.ts`) and must be kept in step manually.
- **On-screen deck:** `TemplatedDeck` renders every section for the chosen theme; `DeckWorkspace` provides a slide rail with live thumbnails, full-canvas editing, theme switching and full-screen view.
- **In-place editing:** click any text to edit through a floating editor (`FloatingTextEditor` , `setByPath`); saves via `PUT /projects/{id}` with `template_data` , which the server re-validates and merges section-by-section so a bad section keeps its last good value.
- **Export path 1 — browser (default button):** each slide is rasterised at 2× (2560×1440) with bounded concurrency of 4, fonts embedded once, and dropped full-bleed onto a 16:9 PPTX with transparent clickable rectangles over e-mail, phone and LinkedIn links. A progress toast is shown. *Trade-off:* slides are images and therefore **not editable in PowerPoint**.
- **Export path 2 — server:** `GET /presentations/{id}/download` re-renders on demand from stored content with the project’s current theme using `python-pptx` (editable text), falling back to the last stored file if re-rendering fails. Per the project log (#52) the server renderer maps the later theme families back to the default Meridian look (“accepted”).
- **Gallery slide:** founder-uploaded photos replace placeholders in both preview and PPTX.

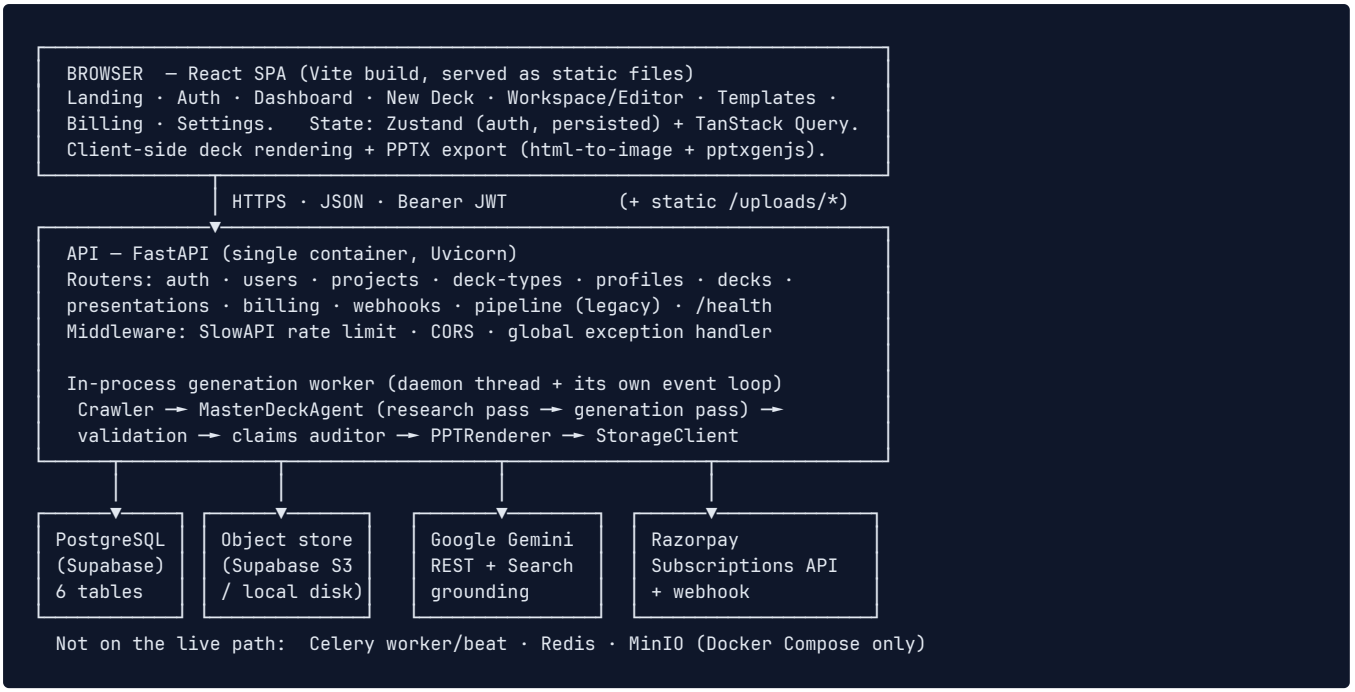
3.10 Billing (`/billing`)

- Plan listing: Free (₹0, 3 decks/month, basic templates, PPTX export), Pro (₹2,500/month, 50 decks, premium templates, “PPTX + PDF export”, priority AI), Enterprise (custom).
- Checkout: server creates a Razorpay subscription (120 billing cycles), the browser opens Razorpay’s hosted checkout (script loaded on demand from `checkout.razorpay.com`), and the signed result is POSTed to `/billing/verify` , which validates the HMAC before upgrading the account.
- Cancellation: cancels at cycle end; access continues until the period ends; the webhook downgrades when Razorpay reports the subscription ended.
- Webhook `/webhooks/razorpay` : HMAC-verified; activates on `subscription.activated|charged|resumed` , ends on `cancelled|halted|completed|expired` .

---

4. Architecture

4.1 Logical architecture



4.2 Backend module map

Package	Responsibility
`app/main.py`	App factory; logging config (`diagnose=False` to keep secrets out of tracebacks); Sentry init; lifespan (create tables, reconcile orphaned jobs); CORS; `/uploads` static mount; `/health` ; global exception handler.
`app/core/`	`config.py` (settings + production guard), `ratelimit.py` , `money.py` (INR/USD formatting, Indian digit grouping, currency guidance for prompts), `genai_client.py` (Gemini REST wrapper).
`app/db/`	`session.py` (engines, per-event-loop NullPool engines for background threads, schema-drift check, `init_db`), `supabase.py` (unused client).
`app/models/`	ORM models: `User` , `CompanyProfile` , `Project` , `Presentation` , `Job` , `Subscription` ; platform-independent `JSONType` .
`app/api/`	`deps/auth.py` (JWT bearer dependency) and `v1/endpoints/*` (Appendix E).
`app/decks/`	`registry.py` (14 deck types, brief fields, intake-field mapping, stage overrides), `sections.py` (65 section specs with typed models and caps), `prompt.py` (section-driven prompt assembly).
`app/profile/`	`parameters.py` (924-parameter catalogue), `relevance.py` (R/C/O/A/N matrix), `values.py` (provenance, merge, derive, completeness), `prompt.py` (profile → prompt block).
`app/agents/`	`master/` (live two-pass agent), `audit/` (claims auditor), and legacy `research/` , `analysis/` , `content/` agents used by the legacy pipeline.
`app/services/`	`crawler/` (Playwright BFS), `branding/` (extractor), `images/` (fetch + normalise), `storage/` (S3 client, upload helpers).

Package	Responsibility
`app/ppt/engine/`	`renderer.py` (1,239 lines), `themes.py`, `template_schema.py`.
`app/workers/`	Celery app and a legacy analysis task (not on the live path).

4.3 Data model

```
users 1──* projects *──1 company_profiles (nullable, ON DELETE SET NULL)
users 1──* company_profiles
users 1──1 subscriptions
projects 1──* presentations
projects 1──* jobs
```

Table	Key columns	Notes
`users`	`id` (UUID string), `email` (unique), `name`, `hashed_password`, `role`, `is_active`, `is_verified`, `stripe_customer_id`, timestamps	`stripe_customer_id` is a leftover from an earlier Stripe design; `is_verified` is never set.
`projects`	`id`, `user_id`, `name`, `company_url`, `status` (draft/analyzing/researching/generating/ready/error), `branding_data` (JSON), `research_data` (JSON), `deck_content` (JSON), `deck_type`, `profile_id`, `template_key`, `assets` (JSON), `is_deleted`, `deleted_at`, `error_message`	Founder inputs, AI output and audit all live in JSON blobs; flexible but not queryable/constrained. Soft delete only.
`company_profiles`	`id`, `user_id`, `name`, `values` (JSON: `{param: {value, source, source_url, method, confidence, as_of}}`)	~924 sparse parameters; a JSON blob by design.
`presentations`	`id`, `project_id`, `user_id`, `file_url`, `file_size_bytes`, `status`, `format`, `slide_count`, `theme_id`	One row per generation; thumbnail fields unused.
`jobs`	`id`, `project_id`, `celery_task_id`, `status`, `current_step`, `step_progress`, `total_progress`, `message`, `error`, `result`, timestamps	Drives the progress UI.
`subscriptions`	`id`, `user_id` (unique), `stripe_subscription_id`, `razorpay_subscription_id`, `plan`, `status`, period start/end, `projects_used_this_month`, `exports_used_this_month`	Usage counters exist but are never written (B-01).

Schema management: `Base.metadata.create_all()` at start-up (creates missing tables only), a hand-maintained idempotent `scripts/db/init.sql`, a runner `scripts/db/migrate.py` (`--check` reports drift), and a start-up **schema-drift check** that logs an explicit error naming missing columns. There is **no Alembic** despite the dependency listing it.

4.4 Concurrency model

Generation is launched from the request handler into a **daemon** `threading.Thread` that creates its own asyncio event loop. Because an asyncpg connection is bound to the loop that opened it, the database layer provisions a separate **NullPool engine per background loop** and disposes it on exit; the request path keeps its shared pool (size 5, overflow 2, sized for Supabase's connection limit, with statement caching disabled for PgBouncer compatibility). This design was reached after production incidents recorded in the log (entries #21 and the "different loop" crash) and is carefully documented in code, but it carries the operational limits described in finding O-01.

4.5 Configuration

All configuration is environment-driven (`pydantic-settings`). The production boot guard (`core/config.py:138`) enforces: non-placeholder `APP_SECRET_KEY` and `JWT_SECRET_KEY` , JWT key ≥ 32 characters, `APP_DEBUG` false, `GEMINI_API_KEY` and `GEMINI_MODEL` present, and no `localhost` in `CORS_ORIGINS` . It does **not** require storage credentials, Razorpay settings or `DATABASE_URL` . Full variable catalogue in Appendix F.

4.6 Deployment topology — as documented vs. as evidenced

Aspect	What the repository evidences
Frontend (live, per log #61)	Firebase Hosting, project `pitch-deck-e1d6c` , auto-deployed by GitHub Actions on pushes to `main` that touch `ipreneur/frontend/**` . `VITE_API_URL` is baked at build time in the workflow.
Backend (live, per workflow comment and log)	Cloud Run (`…asia-northeast1.run.app`), built by a Cloud Build trigger watching `Infopace/Pitch-Deck` . The trigger configuration is **not in the repository**.
Database / storage	Supabase Postgres (`ap-northeast-1`) and Supabase Storage (S3 API), per log and `render.yaml` .
Alternative targets in repo	`render.yaml` (states in its own header that the live API “was created by hand … applying this blueprint would create a second service”); `wrangler.jsonc` (Cloudflare Worker static assets); `vercel.json` . The log shows the frontend has moved Vercel → Cloudflare → Firebase.
Containers	Backend image: `python:3.11-slim` , Chromium dependencies, non-root user `appuser` (uid 1001), Playwright pinned to 1.52.0, `CMD uvicorn … --port ${PORT:-8000}` . Frontend image: multi-stage Node 20 → nginx; its production stage copies `../nginx/nginx.conf` , which does not exist in the repository and is outside the Docker build context (finding O-07).

5. Interfaces

5.1 External interfaces

Counterparty	Direction	Data sent / received	Safeguard in code
Google Gemini (`generativelanguage.googleapis.com`)	Outbound HTTPS	Crawled website text, founder-entered facts (including financial figures and founder contact details), profile parameters → generated deck JSON and grounding sources	API key in `x-goog-api-key` header (not URL); timeout 180 s; retry on 429/5xx
Target company websites	Outbound (headless Chromium)	Page requests → HTML	Same-domain restriction for link following only (**no** SSRF protection — S-02)
Arbitrary image URLs (logo)	Outbound `httpx`	Image bytes	8-second timeout; redirects followed to any host (S-02)
Razorpay	Outbound API + inbound webhook + browser script	Subscription creation, payment signature	Basic-auth API keys from environment; HMAC verification; keys never sent to browser except public key id
Supabase Postgres / Storage	Outbound	All application data; generated PPTX	TLS (pooler URL); credentials from environment
Sentry (optional)	Outbound	Error events	`send_default_pii=False` ; traces sampled at 10 %
Google Fonts	Browser	Font CSS	Embedded once per export to avoid repeated fetches
Unsplash	Outbound (`image_service.fetch_unsplash_background`)	Keyword query	Function exists; not called on the live path (it targets Unsplash's keyword "Source" endpoint, which Unsplash is understood to have deprecated — treat as dead code; verify before reuse)

5.2 API summary

The API is versioned under `/api/v1` . Appendix E lists all endpoints with authentication and rate-limit status. Responses use a mixed convention (some wrapped `{success, data}` , some bare models); the browser client recursively camel-cases response keys, which has caused several defects recorded in the log (for example `total_progress` vs. `totalProgress`).

6. Security Assessment

6.1 Control summary

Domain	Control present	Assessment
Authentication	bcrypt (salted) password hashing; HS256 JWT with `exp`; active-user check on every request	Adequate baseline; gaps listed in S-05, S-07
Authorisation	Every project, deck, presentation and profile query filters by `user_id`; soft-deleted rows excluded	Good, with one gap (S-04)
Brute-force protection	`10/minute` per IP on login/register; `60/hour` create; `20/hour` analyze	Partial; in-memory, per instance, IP identity unverified (S-06)
Transport security	HTTPS terminates at platform; CORS allow-list with optional anchored regex	Adequate; no HSTS/CSP defined by the app (S-05)
Secrets management	Environment variables; `.env*` git-ignored; WIF in CI (no stored cloud key); production boot guard	Good design; execution undermined by S-03
Input validation	Pydantic on all bodies; URL type validation; per-section AI-output validation with caps	Good; unbounded free-form JSON in two places (S-09)
Output encoding	React escapes text; no `dangerouslySetInnerHTML` / `innerHTML` anywhere in `src/`; contact links normalised to `mailto:`, `tel:` or `https://`	Good
Payment security	Signature verification (checkout and webhook), constant-time compare, role protection	Good
Logging	Loguru with `diagnose=False` (no local-variable dumps); Sentry PII off; generic 500 body	Good; one exception (S-10)
Container hardening	Non-root user; slim base	Adequate
Error handling	Global handler returns a generic body	Good; see S-10
Audit trail	None (no user-activity log)	Gap (O-04)

6.2 Findings

S-01 — Unauthenticated, unthrottled, cost-bearing endpoint · High

Evidence. `ipreneur/backend/app/api/v1/endpoints/pipeline.py:26` — `POST /api/v1/pipeline/generate(intake)` has no `Depends(get_current_user)` and no `@limiter.limit`. It crawls the supplied site (headless Chromium, up to 18 pages), then runs branding extraction, research and analysis — three Gemini calls, one with an 8,192-token thinking budget. The router is mounted in `routes/__init__.py` and is therefore live in production; interactive docs are disabled in production but the route is not.

Impact. Any internet user can spend the company's Gemini quota and Chromium capacity at will, exhaust the instance's memory (512 MB on the documented free plan), and use the server as an outbound request relay (see S-02). There is no per-user attribution.

Recommendation. Remove the endpoint (it is a pre-production prototype superseded by `projects/{id}/analyze`), or require authentication, apply `GENERATION_LIMIT`, and enforce the same quota as the main path.

S-02 — No SSRF protection on crawler and image fetcher · High

Evidence. `services/crawler/web_crawler.py:84` (`_is_crawlable`) only compares the host with the seed host; the seed URL itself is never validated. `ProjectCreate.company_url` is a Pydantic `HttpUrl`, which accepts `http://127.0.0.1`, `http://10.x.x.x`, `http://169.254.169.254/` and `http://localhost:…`. Playwright follows redirects. `services/images/image_service.py:62` fetches arbitrary logo URLs (which originate from crawled pages or model output) with `follow_redirects=True` and no address check.

Impact. An authenticated user (or, via S-01, anyone) can make the server request internal services or the cloud metadata endpoint. Response content reaches the model prompt and, for images, the generated deck, creating a data-exfiltration path. On Google Cloud this class of flaw can expose service-account tokens.

Recommendation. Resolve the hostname and reject loopback, private, link-local, multicast and metadata ranges **before** connecting and **after every redirect**; restrict schemes to `http/https`; restrict ports; apply the same guard to image fetches; consider running the crawler in an egress-restricted network.

S-03 — Credentials in version control · High

Evidence.

- `ipreneur/backend/_test_step1.py`, `_test_step3.py`, `_test_step4.py`, `_test_step5.py` contain eight occurrences of two 39-character strings in the Google API key format (`AIza…`), assigned to `KEY1` / `KEY2`. Whether the keys are still valid cannot be determined from the repository; they must be treated as compromised. (The values are deliberately not reproduced in this report.)
- Project log entry #30 records that real credentials (Supabase database password, anon key, S3 access and secret keys) were committed in `.env.example` at commit `e346ba6`, removed from the file in a later commit, but that the secrets “remain in git history … and should be rotated and history scrubbed.” Entry #52 (session of 25 Sep 2026) lists this as still outstanding. That commit is not present in the clone audited, so history rewriting may have occurred or the history was re-based — **the rotation status could not be verified**.

Recommendation. Revoke and re-issue the Google keys and every credential named in entry #30; delete the key-bearing scripts (they target a retired model and are obsolete); scrub history with `git filter-repo`/BFG and force all clones to re-sync; enable GitHub secret-scanning push protection; record the rotation in the log.

S-04 — Cross-tenant profile attachment (IDOR) · Medium

Evidence. `projects.py:378` stores `payload.profile_id` without checking that the profile belongs to the caller, and the generation task loads it with `db.get(CompanyProfile, _profile_id)` (`projects.py:88`) with no owner check. The profile endpoints themselves are correctly scoped (`_owned()` in `profiles.py`).

Impact. A user who learns another user’s profile UUID can cause that profile’s parameters (potentially financials, cap table, founder details) to be injected into a deck they own and then read. UUIDs are not guessable, and the current web UI never sends `profile_id` (the path is reachable only by calling the API directly), which both limit likelihood — but the flaw will become live as soon as the profile UI ships, and identifiers can leak through logs, support exchanges or shared links.

Recommendation. Validate `profile.user_id == current_user.id` at project creation and again inside the task; add a regression test.

S-05 — Token storage and absence of browser hardening · **Medium**

Evidence. `frontend/src/stores/authStore.ts:81` persists `accessToken` to `localStorage`. There is no refresh endpoint (the store calls `/auth/refresh`, which does not exist, though the call is never triggered), logout is a client-side no-op server-side, and tokens cannot be revoked before the 30-minute expiry. `firebase.json` defines no `headers` block, so the SPA is served without CSP, `X-Frame-Options` / `frame-ancestors`, `X-Content-Type-Options`, `Referrer-Policy` or HSTS from the application layer.

Mitigating factors. No HTML-injection sinks were found in the frontend; contact links are normalised; the API origin differs from the app origin.

Recommendation. Add response headers via `firebase.json`, with a CSP that allows only the required origins (Razorpay, Google Fonts, the API); move to `HttpOnly; Secure; SameSite` cookies or short-lived access tokens with rotating refresh tokens and a server-side denylist; implement real logout.

S-06 — Rate limiting depends on an unverified client identity · **Medium (verify in deployment)**

Evidence. `core/ratelimit.py` keys limits on `get_remote_address` and stores counters in process memory. The container starts Uvicorn without proxy-header trust configuration, and the application is deployed behind Google's front end.

Impact. If Uvicorn does not trust `X-Forwarded-For`, all clients appear to come from the proxy address: the `10/minute` login limit would then be **shared by every user** (self-inflicted lockout) rather than per attacker. With several instances, limits multiply. The file's own docstring acknowledges the per-instance limitation.

Recommendation. Configure `--proxy-headers` with the platform's trusted proxy range, key limits additionally on account identifier for auth endpoints, and move storage to Redis once more than one instance runs.

S-07 — Account-security hygiene · **Medium**

Evidence. `api/v1/endpoints/auth.py`: `RegisterRequest.password` is an unconstrained `str` (no minimum length or strength rule); registration returns HTTP 409 "Email already registered" (account enumeration); there is no e-mail verification (column `is_verified` is never updated), no password-reset flow, no lockout or MFA, and no admin tooling to deactivate users other than a database edit. `bcrypt.hashpw` on passwords above 72 bytes raises in current bcrypt releases, which would surface as an HTTP 500.

Recommendation. Enforce a minimum length (12+) with a breached-password check, return a neutral response on duplicate registration, add verification and reset flows, cap password length at the bcrypt limit with a clear validation error.

S-08 — Upload handling · **Medium**

Evidence. `services/storage/assets.py`: the allow-list is applied to the **client-supplied** `Content-Type` header; SVG (`image/svg+xml`) is permitted; there is no magic-byte or decode check; the whole file is read into memory before the 8 MB check; no limit on cumulative uploads per project or per user; the upload route has no rate limit; files are written to `backend/uploads/<project>/…` on the instance disk and served by an **unauthenticated** `StaticFiles` mount (`main.py:183`).

Impact. Executable SVG (script) can be hosted on the API origin; spoofed content types bypass the filter; large or numerous uploads can fill the disk or memory; on ephemeral hosting, assets disappear on every deploy or sleep (the `render.yaml` header documents this for PPTX files, and uploads share the weakness).

Recommendation. Validate by decoding with Pillow, drop SVG or sanitise it, serve user content from a separate cookie-less domain/bucket with `Content-Disposition` / `X-Content-Type-Options: nosniff`, stream with a hard size limit, and store in object storage with signed URLs.

S-09 — Unbounded free-form JSON · Low

Evidence. `ProjectCreate.branding_data: dict[str, Any]`, `ProjectUpdate.template_data: dict[str, Any]` (partially bounded by section caps after validation) and `ProfileCreate.values: dict[str, Any]` carry no size limit; FastAPI applies no body-size cap by default.

Recommendation. Enforce a request-body limit at the proxy and in middleware; cap key counts and string lengths for `branding_data`.

S-10 — Internal error text returned to clients · Low

Evidence. `projects.py:400` returns `detail=str(e)` on a 500. Job failures store the raw exception string (which can contain upstream API error text) in `jobs.error` and `projects.error_message`, and the status endpoint returns it to the user.

Recommendation. Map failures to a short list of user-safe codes/messages; keep detail in logs and Sentry.

S-11 — Download filename built from user input · Low

Evidence. `presentations.py:130,161` interpolates `project.name` into the `Content-Disposition` header unescaped. Names containing quotes, control characters or characters outside Latin-1 (for example Devanagari) can break header encoding or confuse clients; the first failure is swallowed by the surrounding `except` and the fallback then repeats it.

Recommendation. Use RFC 6266/5987 encoding (`filename*=UTF-8''…`) after sanitising.

S-12 — Dependencies and supply chain · Medium

Evidence. `backend/requirements.txt` pins only `playwright==1.52.0`; every other package floats. There is no backend lock file. It lists libraries the code does not use (`google-generativeai`, `google-genai` — the client uses raw REST; `passlib[bcrypt]` — the code calls `bcrypt` directly; `aiosqlite`, `alembic`) and omits `supabase` although `app/db/supabase.py` imports it (harmless only because nothing imports that module). `python-jose` has had published vulnerabilities in earlier releases and is weakly maintained. The deploy workflow uses `npx firebase-tools@latest` and actions pinned by tag, not commit SHA. No dependency-audit or secret-scanning step exists. Frontend dependencies use caret ranges with a lock file present.

Recommendation. Generate a hashed lock file (`pip-compile` / `uv`), remove unused packages, replace `python-jose` with `PyJWT` or `authlib`, pin tooling versions and action SHAs, and add `pip-audit`, `npm audit`, Dependabot and secret scanning to CI.

S-13 — Hard-coded dev defaults · Informational

`config.py` defaults (`minioadmin`, `ipreneur_dev`, `change_me`) and `docker-compose.yml` development credentials are acceptable for local use and are caught by the production guard for the secrets that matter; the guard does not, however, check storage credentials, so a production deployment with missing S3 settings silently falls back to local disk (see O-03).

6.3 Positive security observations

- Production start-up guard with an aggregated, readable failure message.
- Interactive API docs and ReDoc disabled when `APP_ENV=production`.
- `diagnose=False` on Loguru — a deliberate decision documented as preventing API keys from appearing in tracebacks.
- Gemini key sent in a header, never in the URL.
- Tenant isolation implemented consistently in the query layer rather than relying on UI hiding.
- Path-traversal guard in `read_asset_bytes` (resolved path must remain inside the upload directory).

- Webhook rejects requests when the secret is unset (HTTP 503) rather than accepting unsigned calls.
- Cloud deploy from GitHub uses short-lived OIDC federation scoped to the repository, avoiding a stored service-account key — and the log documents that the organisation policy forbids key creation.
- Soft-deleted projects are excluded from all reads.

---

7. Data Governance, Privacy and Compliance

7.1 Data inventory

Data class	Examples	Where stored	Sensitivity
Account data	Name, e-mail, bcrypt hash, role	`users`	Personal data
Business confidential	ARR/MRR, burn, runway, CAC/LTV, cap table, fundraising ask, strategy, risks	`projects.branding_data` , `company_profiles.values` , `deck_content`	**High** commercial sensitivity
Third-party personal data	Founders'/team members' names, titles, e-mail, phone, LinkedIn	Same JSON blobs	Personal data of people who may not be users
Public web content	Crawled pages, competitor facts, market data with source URLs	`research_data`	Low (public), but copyright/ToS considerations
User assets	Logos, gallery photos	`backend/uploads` (disk)	May contain personal likenesses
Generated files	PPTX decks	Object storage / `backend/storage`	High (contain the above)
Payment data	Razorpay subscription IDs only; no card data handled by the application	`subscriptions`	Low (PCI scope minimised by hosted checkout)
Telemetry	Error events (Sentry, optional)	Sentry	Low (PII off)

7.2 Findings

D-01 — No data-subject lifecycle · **Medium**

Projects are soft-deleted (`is_deleted` , `deleted_at`) and never purged; generated PPTX files, uploaded assets, `jobs` and `presentations` rows are not removed on deletion; profiles are hard-deleted but their derived content remains in decks. There is no account-deletion, data-export (portability) or retention endpoint, and no scheduled cleanup (Celery Beat is configured only in Compose with no scheduled tasks).

Recommendation. Define retention periods; implement hard-delete jobs for soft-deleted projects (including storage objects and uploads); add account deletion and export; document backups' retention window.

D-02 — Ownership and notice inconsistencies · **Low**

Root README: "© Infopace Management Pvt. Ltd."; `ipreneur/README.md` : "iPreneur Inc."; product name "Mint" in the UI and configuration vs. "iPreneur" in code, the Razorpay checkout and `description="Pro plan"` . There is no Terms of Service, Privacy Policy or cookie/consent text in the frontend routes.

Recommendation. Align legal entity and product naming; publish ToS and Privacy Policy and link them at registration and checkout.

D-03 — Customer-style and bulky artefacts under version control · **Medium**

Evidence (tracked files).

- `ipreneur/backend/storage/` — **55** generated `presentation.pptx` files, one per project UUID.

- `ipreneur/backend/uploads/` — **21** uploaded logos and gallery images across 5 project folders.
- `ipreneur/backend/pipeline_output.json`, `pipeline_test_output.json`, `pipeline_test_output_repeat.json` — scraped and generated analysis output.
- 2 further `generated_presentation_*.pptx`, `growth.pptx`, `celerybeat-schedule` (binary), seven install/log text files.
- Seven binary design archives at the repository root (`*.zip`) and a design handoff HTML.
- 31 files under `.claude/skills/` (third-party assistant skill definitions, locked in `skills-lock.json`) — tooling, not product.

Repository size is 41 MB packed (58 MB of working-tree content). If any of these files derive from real user sessions (the UUID filenames and uploaded logos suggest that they do, or are real-company test cases), they constitute personal or confidential data in a source repository with a wider access list than the production database. `.gitignore` excludes `.env`, `*.db` and logs but not `storage/`, `uploads/` or generated decks.

Recommendation. Remove from the tree and from history; add `storage/`, `uploads/`, `*.pptx`, `pipeline_*output*.json` and logs to `.gitignore`; move design archives to an artefact store.

D-04 — Third-party AI processing · **Medium**

Business-confidential and personal data (including founders' phone numbers and e-mail addresses when entered) are transmitted to Google's Gemini API, with Google Search grounding on research calls. Nothing in the repository records the data-processing agreement, the Gemini tier in use (consumer/free keys versus paid/Vertex terms differ materially on data retention and model training), user consent, or a sub-processor list. The legacy research agent also issues DuckDuckGo queries containing company names.

Recommendation. Confirm the commercial/enterprise Gemini terms (no training on customer data), execute a DPA, show a clear AI-processing notice at intake, and let users exclude personal fields (founder phone/e-mail) from the model prompt (they are only needed for the final contact slide).

D-05 — Crawling conduct · **Low**

The crawler does not read or honour `robots.txt`, identifies as a generic browser, and fetches up to 18 pages from a third-party site on a user's request. Users can point it at any site, including one they do not own.

Recommendation. Honour `robots.txt` and rate-limit per host; use an identifiable user-agent with a contact URL; add a Terms clause that users must have the right to submit the URL.

7.3 Regulatory mapping (indicative, not legal advice)

Regime	Why it may apply	Observed state
India — Digital Personal Data Protection Act, 2023	Indian operating entity; Indian founders and their team members' personal data	No consent, notice, grievance-officer, erasure or breach-notification mechanisms evidenced
EU/UK GDPR	If EU/UK residents register or their data are entered	Same gaps; plus international transfer terms for Google and Supabase
PCI-DSS	Subscription payments	Card data never touches the application (Razorpay hosted checkout, SAQ-A-style scope); no PCI artefacts in repo
IT Act 2000 / CERT-In directions	Indian service provider; incident reporting (6-hour) and log-retention (180-day) expectations	No incident-response plan or log-retention configuration evidenced

Regime	Why it may apply	Observed state
Consumer-protection / advertising standards	Marketing claims on the landing page (“Every number comes with its reasoning attached”, “audit pass”)	Audit findings are computed but not shown to users (Q-04)
Intellectual-property terms	Crawled website content and Unsplash/third-party imagery	No user warranty/licence terms evidenced

Certifications. None (SOC 2, ISO 27001, etc.) is claimed or evidenced in the repository. Appropriately, the claims auditor itself flags unevidenced certification claims in *generated* decks — the product holds its output to a standard that the product’s own marketing site and documentation should also meet.

---

8. AI-Specific Risk Assessment

Risk	Where it arises	Existing control	Residual risk / recommendation
Fabricated facts (hallucinated revenue, customers, certifications)	Generation pass	Prompts forbid invention; founder figures marked as ground truth; section schemas; deterministic claims auditor; founder-provided team overrides web search	Auditor is advisory and unseen by users (A-01). Make findings visible, optionally block export on `error` severity, and require an "I have verified this deck" acknowledgement.
Indirect prompt injection	Crawled page text, competitor snippets and founder text enter the prompt	Output is schema-validated and rendered as text (no HTML)	A malicious page could steer the deck's wording, add misleading claims or links. Delimit and label untrusted content in prompts, strip instruction-like text, and keep human review in the loop.
Same-name confusion (research attaches a different company's founders)	Research pass	Founder-provided team replaces search results; log documents the "Nimbus" incident	Retained for website-less projects; maintain.
Stale model dependency	`GEMINI_MODEL`	Configurable; log #94db209 records an outage when Google retired `gemini-2.5-flash`	Add a start-up model-availability probe and alerting; keep a fallback model. `config.py` declares a `groq_api_key` setting and `.env.example` lists `ANTHROPIC_*` variables, yet **no code uses either** (documentation drift).
Truncated or malformed output	Large decks (24–25 slides)	Truncation detection; lenient JSON repair; retries; per-section isolation	Good. Monitor truncation rates.
Non-determinism / reproducibility	Temperature 0.7	Deck content stored after generation; regenerate creates a new version	Store prompt hash, model name and timestamp per generation for audit reproduction (not currently stored).
Cost	Every generation = research (grounded) + generation (up to 32 K output tokens) + team search	20 generations/hour/IP	Unenforced plan quotas (B-01) and S-01 undermine this. Add per-user and global budgets and spend alerts.
Bias / fairness / harmful content	Free-text generation	None specific	Low for business decks; add output moderation if deck sharing is introduced.
Currency integrity	Mixed ₹/$ outputs	`money.py`, prompt guidance, auditor flags wrong-currency symbols	Strong; field names retain the legacy `_usd` suffix for compatibility (documented).

A-01 — Audit results not exposed · **Medium**

See Q-04: the deck-level `audit` object is stored (`master_agent.py:415-427`) but nothing in the frontend reads it.

A-02 — No generation provenance record · Low

Model name, prompt version, research sources used and timing are not persisted per generation beyond `research_data` content and job messages, limiting after-the-fact review of a disputed deck.

---

9. Billing and Financial Controls

Control	State
Pricing display vs. actual charge	Price text in `/billing/plans` is display-only; the actual amount is whatever the Razorpay plan behind `RAZORPAY_PRO_PLAN_ID` bills. The log (#59) records a near-miss where "Rs 29" nearly replaced "$29" — now ₹2,500 — showing the display string is a manual-sync risk.
Payment verification	Checkout result validated with HMAC(`payment_id|subscription_id`) using the key secret; the subscription must belong to the caller. Correct for Razorpay subscriptions.
Webhook	HMAC over raw body, constant-time compare, secret mandatory. Handles activation, renewal and end events. `payment.failed` / `pending` and refunds are not handled; handlers are idempotent in effect (setting state) but there is no event-ID de-duplication or ordering protection.
Role changes	Only `free` ↔ `pro` ; `enterprise` / `admin` never altered by payment events.
Entitlements	**Not enforced** (B-01).
Production readiness	Per log #59, Razorpay key id and plan id were unset in production at 27 Oct 2026, i.e. checkout returned HTTP 503; this must be confirmed before launch.
Tax / invoicing	Not addressed in code (GST invoicing, tax display) — outside repo scope but relevant to financial audit.
Refund/dispute handling	Not present.

B-01 — Plan limits are not enforced · **High**

Evidence. `billing.py:58` defines `PLAN_LIMITS` and the API reports it, and `Subscription.projects_used_this_month` / `exports_used_this_month` exist (`models.py:306`), but no endpoint reads or increments them — a search of `app/` finds no other references. `create_project` , `trigger_analysis` and the download endpoint contain only per-IP rate limits.

Impact. The Free plan's headline restriction ("3 pitch decks/month") is not a restriction; Pro's 50/month likewise. Each unrestricted generation consumes paid AI capacity. This is both a revenue-leakage and a cost-control weakness, and a potential mis-statement to customers.

Recommendation. Enforce at project creation/analysis and at export; increment in the same transaction; reset on the billing period; return a clear 402/403 with an upgrade prompt; add tests. Until implemented, remove or soften the marketed limits.

B-02 — Marketed features without implementation · **Medium**

"PPTX + PDF export" (Pro), "Priority AI processing" (Pro), "API access", "Team collaboration" and "Custom branding" (Enterprise) appear in plan copy (`billing.py:90-113`). The export endpoint accepts `format=pdf` but only ever returns PPTX; there is no priority queue, API-key system or multi-user workspace.

Recommendation. Align marketing with delivered functionality or schedule delivery; unfulfilled paid features create refund and misrepresentation exposure.

10. Quality Assurance

10.1 Automated tests — executed for this audit

Counts in brackets are test *functions*; parametrised cases expand them to 559 executed tests in total.

Module	Focus	Result
`test_deck_registry.py` (22)	Registry integrity, intake-field mapping, stage overrides	Pass
`test_deck_registry_frontend_sync.py` (5)	Python and TypeScript deck/section definitions stay identical	Pass
`test_profile.py` (37)	Parameter catalogue, relevance matrix, provenance merge, derive, completeness	Pass
`test_profile_api.py` (12)	Profile CRUD, readiness, merge semantics, founder-over-crawl precedence	Pass (imports the Celery module)
`test_claims_auditor.py` (17)	Auditor checks	Pass
`test_renderer_sections.py` (11)	Section → slide rendering	Pass
`test_appendix.py` (12)	Dynamic appendix	Pass
`test_deck_prompt_currency.py` (6)	INR/USD prompt integrity	Pass
`test_schema_drift.py` (8)	Drift detection, including “unreachable DB is reported as unknown, never as clean”	Pass
Total (offline)		**559 passed, 1 skipped, 0 failed** (4.95 s)
`test_crawler.py`, `test_pipeline.py`	Live-website and live-model scripts	**Not runnable offline** — they execute at import time, launch Chromium and call the network, and abort pytest collection when no browser is present

Environment note: the tests were run with dependencies installed ad hoc (the repository ships no backend lock file or `pytest.ini`); a first run without `celery`, `asyncpg` and `greenlet` failed at import, illustrating Q-02.

10.2 Findings

Q-01 — No continuous-integration gate · **Medium**

The sole workflow (`firebase-hosting-deploy.yml`) builds and deploys the frontend; it runs no tests, linter or type-check as a separate gate (the build runs `tsc`). Nothing blocks a merge to `main` on a failing backend test, and backend deployment is triggered externally. Single-committer history shows no evidence of pull-request review.

Q-02 — Reproducibility of the test environment · Low

No `pytest.ini` / `conftest.py`, no lock file, import-time side effects in two test modules (even after commit `9317366` aimed to stop pytest collecting manual scripts). Developers need undocumented steps to get a green run.

Q-03 — Coverage gaps · Medium

Not covered by automated tests: authentication and authorisation (cross-tenant access), rate limiting, billing and webhook signature paths, upload validation, the crawler's URL handling, the download endpoint, the background-job lifecycle, and the **entire frontend** (Vitest is configured; zero test files exist; there is no ESLint configuration file although `npm run lint` is defined; TypeScript `strict` is on, with `noUnusedLocals` / `noUnusedParameters` off).

Q-04 — Advertised audit not surfaced · Medium

The claims-audit report is computed and stored but is not displayed. The landing page headline "Every number comes with its reasoning attached" and the "audit pass" description therefore over-state what the user can see.

Q-05 — Manual duplicate definitions · Low

Themes exist in two languages (`themes.py` , `themes.ts`); section definitions are guarded by the sync test, but theme parity is a documented manual obligation.

10.3 Defects recorded and corrected (from project log) — evidence of an active fix loop

Selected examples: blank workspace crash from a temporal-dead-zone reference (#1); progress-bar `undefined%` from camel-casing (#4); step-name mismatch between frontend and backend (#5); "Future attached to a different loop" crash on every analysis (#21); misconfigured thinking budget yielding "Parsed deck JSON has no slides" (#22); silent validation failure on textareas (#24); invented/fabricated dashboard metrics removed (#49); uploaded logo never appearing in downloads (#37 / `presentations.py` comment); Pro price typo (#59); Overview page stale after editing elsewhere (commit `ac32b54`); market-chart crash that killed whole-deck generation (`6b66650`); USD figures appearing on INR projects (`af13802`); slow PPTX export root-caused to repeated font-CSS fetches (`2c3a010`).

---

11. Operations, Resilience and Maintainability

O-01 — Generation runs in daemon threads · **Medium**

`projects.py:356` starts a `daemon=True` thread per run. Consequences: an instance restart, deploy or idle-sleep kills in-flight work (mitigated by marking those jobs failed at the next start); there is no queue depth, concurrency cap, back-pressure or retry; each run holds a Chromium instance and a long LLM call in the web process, so a handful of concurrent users can exhaust memory (the Render blueprint itself warns of 512 MB limits and OOM); and the design cannot scale beyond one instance without duplicating limits. The Celery code that would solve this describes an older four-agent pipeline and is deliberately not deployed (documented in `render.yaml`). **Recommendation.** Port the master pipeline to Celery/Cloud Tasks (or Cloud Run Jobs), add a per-user concurrency limit of one active generation, and cap global concurrency.

O-02 — Schema changes are manual · **Medium**

No Alembic. `create_all()` never alters tables; `init.sql` is applied by a script; a start-up check only *warns*. The log records a production outage in which a missing `projects.profile_id` column made the dashboard spin silently. **Recommendation.** Adopt Alembic with versioned migrations applied in the deploy step; fail health checks on drift.

O-03 — Silent storage fallback · **Medium**

`StorageClient` falls back to local disk when S3 settings are absent or an upload fails (`s3_client.py`), and `download` also reads local disk first. In an ephemeral container this produces decks that exist until the next deploy, then 404. The production guard does not require storage settings. **Recommendation.** Fail loudly in production when object storage is not configured; store the object key, not a URL.

O-04 — No audit trail or operational telemetry · **Medium**

There is no security/audit event log (logins, plan changes, deletions, downloads), no admin console, and no metrics. `/health` returns a static `{"status": "ok"}` without checking database or storage reachability, so the platform health check cannot detect a broken database. Sentry is optional and, per `render.yaml`, may be unset. **Recommendation.** Add structured audit events, a deep health endpoint (`/ready`), and basic metrics (generation duration, failure rate, AI cost per deck).

O-05 — Backup and disaster recovery not evidenced · **Medium**

Backups rely implicitly on Supabase's managed service; the project log (#31) notes the free-tier database **auto-paused after inactivity and refused connections**, which is a business-continuity risk for production. No restore test, RPO/RTO or runbook exists in the repository. **Recommendation.** Use a paid tier, document RPO/RTO, test restores, and add runbooks.

O-06 — Documentation drift · **Medium**

Examples (all verifiable against code):

Document	Says	Code does
`docs/architecture/ARCHITECTURE.md`	Claude/Anthropic generates content; Stripe billing; WebSocket channel; refresh tokens (30 days); presigned S3 URLs; 5-page crawl; 12 slides in parallel; 8 layouts; `projectStore` / `editorStore`	Gemini via REST; Razorpay; no WebSocket; no refresh endpoint; public/CDN URLs; up to 18 pages; 5–25 slides in one structured call; only `authStore` exists

Document	Says	Code does
`README.md` / `ipreneur/README.md`	"Gemini 2.5 Flash"; Compose frontend on :3001; phases 6–10 incomplete; 7 deck types in tree comment	`gemini-3.8-flash` after a retirement; Compose maps :3000; most phases done; 14 deck types
`FIX_SUMMARY.md`	Describes `import google.generativeai as genai` and `asyncio.ensure_future`	Client is raw REST; background work is a thread
`.env.example`	`ANTHROPIC_*` variables	No code consumes them (also an unused `GROQ_API_KEY` setting)
`render.yaml`	Render is the backend host	Live backend is Cloud Run; file warns it does not track the live service

Recommendation. Treat `PROJECT_LOG.md` plus a freshly written architecture document as the sources of truth; delete or banner obsolete files.

O-07 — Container and infrastructure artefacts that cannot work as written · Low

The frontend production Dockerfile stage `COPY ../nginx/nginx.conf` references a path outside the build context and no `nginx` directory is tracked; the Terraform/Kubernetes configuration mentioned in the root README does not exist in the tree. `docker-compose.yml` sets `command: … --reload` and mounts source for the backend, i.e. it is development-only, and its default Postgres/Redis passwords are published values.

O-08 — Repository hygiene · Low

Debug and one-off scripts (`_test_step*.py` , `check_*.py` , `inline_run.py` , `run_generate_deck.py` , `download_playwright_headless.py`), installation logs and a stray root `package-lock.json` are tracked. `ProjectCard.tsx` and `NewSidebarLeafGraphic.tsx` are unused (log #52). The `db/supabase.py` client, `workers/` , legacy agents, `pipeline.py` , Unsplash helpers and the `stripe_customer_id` column are dead or legacy code that enlarges the attack and maintenance surface.

O-09 — Observability of the AI supply chain · Low

No alerting is wired to Gemini failure, quota exhaustion or model retirement beyond logs; the log shows a model-retirement incident discovered only by user failure.

11.1 Positive operational observations

- **Orphaned-job reconciliation** at start-up turns permanent spinners into visible, retryable errors.
- **Schema-drift check** distinguishes "no drift" from "could not check" — a rare and correct design choice.
- **Per-event-loop database engines** documented in detail after a real concurrency defect.
- **Sentry integration** with PII disabled and sampling.
- **Branded failure experiences** (generation failure, 404, crash, offline) so that errors are intelligible to non-technical founders.
- **Project log discipline** — 61 numbered entries recording problem, cause, fix and files changed, including admissions of unfinished work.

12. Risk Register and Remediation Roadmap

12.1 Risk register

ID	Risk	Likelihood	Impact	Rating
S-01	Unauthenticated cost abuse / DoS via `/pipeline/generate`	**High**	**High**	**High**
S-02	SSRF to internal/metadata endpoints	**Medium**	**High**	**High**
S-03	Exposed API/DB/storage credentials	**High**	**High**	**High**
B-01	Unenforced plan quotas (revenue/cost leakage)	**High**	Medium–High	**High**
S-04	Cross-tenant profile attachment	**Low**	**High**	**Medium**
S-05	Token theft via XSS; no revocation; no CSP	**Medium**	**Medium**	**Medium**
S-06	Rate limits keyed on proxy IP / per instance	**Medium**	**Medium**	**Medium**
S-07	Weak account hygiene (no policy/verification/reset)	**Medium**	**Medium**	**Medium**
S-08	Unsafe upload handling	**Medium**	**Medium**	**Medium**
S-12	Unpinned/unaudited dependencies	**Medium**	**Medium**	**Medium**
D-01	No erasure/retention	**High**	**Medium**	**Medium**
D-03	Customer-style data in repository	**Medium**	**Medium**	**Medium**
D-04	Third-party AI processing without documented terms/consent	**Medium**	**Medium**	**Medium**
A-01 / Q-04	Audit invisible; AI inaccuracies reach investors	**Medium**	**Medium**	**Medium**
B-02	Marketed features not delivered	**Medium**	**Medium**	**Medium**
O-01	Thread-based generation fragility	**High**	**Medium**	**Medium**
O-02	Manual schema management	**Medium**	**High**	**Medium**
O-03	Silent local-storage fallback loses decks	**Medium**	**Medium**	**Medium**
O-04 / O-05	No audit trail; DR/backups unproven; DB auto-pause	**Medium**	**High**	**Medium**
Q-01 / Q-03	No CI gate; untested security paths and frontend	**High**	**Medium**	**Medium**
O-06	Documentation drift misleads operators	**High**	Low–Medium	**Medium**
S-09 – S-11, D-02, D-05, O-07 – O-09	Hygiene items	—	**Low**	**Low**

12.2 Prioritised roadmap

Horizon	Actions	Findings closed
Immediate (0–14 days)	Disable or authenticate `/pipeline/generate`. Rotate Google keys and the credentials in log #30; delete `_test_step*.py`; scrub history; enable push protection. Add SSRF guard. Confirm in production: proxy-header trust, Sentry DSN, storage variables, Razorpay variables, Gemini commercial terms.	S-01, S-02, S-03, S-06 (verify), O-03 (verify)
Short term (15–45 days)	Enforce plan quotas and meter exports. Ownership check on `profile_id`. Security headers/CSP; decide cookie vs. token strategy; real logout. Password policy, e-mail verification, reset. Harden uploads (decode, no SVG, separate domain, signed URLs). Remove uncommitted-data artefacts from repo and ignore them. Add CI: backend tests, frontend type-check/build, `pip-audit` / `npm audit`, secret scan; lock and prune dependencies; add Dependabot.	B-01, S-04, S-05, S-07, S-08, S-12, D-03, Q-01
Medium term (45–120 days)	Move generation to a queue with concurrency caps and retries. Introduce Alembic migrations and a `/ready` health check. Implement hard-delete/retention, account deletion and export. Publish ToS/Privacy Policy, AI-processing notice and DPA register. Surface the claims audit in the UI and optionally gate export. Audit-event logging and admin tooling. Backup/restore tests and runbooks; paid database tier. Frontend and API-security test suites (cross-tenant, billing, uploads). Honour `robots.txt`.	O-01, O-02, D-01, D-02, D-04, D-05, Q-03, Q-04, A-01, O-04, O-05
Ongoing	Refresh architecture/README documents from the code; retire legacy modules; align marketed features with delivered ones or build them (PDF, priority processing, Enterprise features); model-availability monitoring; periodic access review.	O-06, O-07, O-08, B-02, O-09

Appendix A — Repository Inventory

Path	Contents
`ipreneur/backend/app/`	FastAPI application (Section 4.2)
`ipreneur/backend/test_*.py`	11 test modules (9 offline-runnable)
`ipreneur/backend/_test_step*.py`, `check_*.py`, `inline_run.py`, `run_generate_deck.py`	One-off debug scripts (S-03, O-08)
`ipreneur/backend/storage/`, `uploads/`	Committed generated/uploaded content (D-03)
`ipreneur/frontend/src/`	React application: 11 pages, 16 shared UI components, workspace/deck-template components (1,488-line `TemplatedDeck`, 802-line `DeckPreview`, 476-line `DeckWorkspace`), landing components, API services, store
`ipreneur/frontend/public/`	Hero and template images, brand SVGs, `offline.html`, `sw.js`
`ipreneur/infrastructure/docker/`	Backend and frontend Dockerfiles
`ipreneur/docker-compose.yml`	Development stack: Postgres, Redis, MinIO, backend, worker, beat, frontend
`ipreneur/scripts/`	`db/init.sql`, `db/migrate.py`, `gen_parameters.py`, `setup/install.sh`
`ipreneur/shared/`	Shared TS constants and types (largely superseded by frontend `types`)
`ipreneur/docs/architecture/ARCHITECTURE.md`	Outdated architecture description (O-06)
`ipreneur/PROJECT_LOG.md`	1,374-line engineering log, 61 entries
`reports/`, `research_notes/`	Deep-research report and six notes on real-world pitch-deck parameters, competitor landscape, India fundraising norms and investor behaviour; includes a self-audit of the product
`.github/workflows/firebase-hosting-deploy.yml`	Frontend CI/CD
`render.yaml`	Render blueprint (non-authoritative)
`.claude/`	Assistant skills/agent definitions (tooling)
Root documents	Bootstrap blueprint (1,049 lines), `FIX_SUMMARY.md`, `Pitch Deck Parameters.docx` (the specification behind the 924-parameter model), design archives, `iPreneur-standalone.html` (single-file demo), `stitch_extracted/` (dashboard design spec)

Appendix B — Deck-Type Catalogue

Key	Label	Slides	Research	Type-specific intake fields
`investor`	Investor / Fundraising	24 (22 for idea and pre-seed)	full	—
`sales`	Sales / Customer	14	light	target buyer*, buyer pain, case studies, pricing notes
`product`	Product / Demo	14	light	persona notes, jobs-to-be-done, demo flow
`partnership`	Partnership / Collaboration	15	full	partner name, what partner brings, the ask
`internal`	Internal / Decision Memo	10	none	initiative, decision requested, problem context, options considered, resources needed, risks
`update`	Investor Update	8	none	period, prior metrics, wins, challenges
`vision`	Vision / Mission	8	full	vision statement, horizon, principles
`general`	General / Company Overview	20	full	—
`elevator`	Elevator / Five Slides	5	light	—
`demoday`	Demo Day	10	full	—
`gtm`	Go-to-Market	22	full	—
`business`	Business Plan	25	full	business purpose, use of funds notes
`master`	Extended / Master	25	full	—
`appendix`	Appendix / Evidence	2 fixed + dynamic	light	— (resolved from the company profile)

* required. Section library: 65 specifications, of which 20 are evidence sections used by the Appendix.

Appendix C — Parameter Groups (924 parameters in 49 groups)

#	Group	Params	#	Group	Params
1	Company Identity	18	26	Legal & Corporate	28
2	Founder & Team	19	27	Evidence & Diligence	21
3	Company Story / Origin	9	28	Contact & Closing	10
4	Problem	28	29	Customer Segmentation	13
5	Customer / ICP	38	30	Positioning	8
6	Solution	13	31	Messaging	10
7	Product	35	32	Acquisition Strategy	15
8	Technology & IP	29	33	Marketing Channels	23
9	Security & Compliance	25	34	Sales Funnel	19
10	Market Opportunity	20	35	Sales Process	10
11	Business Model	31	36	Distribution Strategy	12
12	Traction & Validation	34	37	Customer Retention & Expansion	10
13	Unit Economics	19	38	Geographic Expansion	10
14	Go-To-Market	42	39	GTM Team & Resources	19
15	Sales	41	40	Customer Research	9
16	Operations	13	41	Market Research	8
17	Partnerships & Ecosystem	21	42	Product Architecture	12
18	Competition	21	43	AI / ML — Conditional	17
19	Moat / Defensibility	11	44	Compliance	9
20	Financials	30	45	Customer Breakdown	12
21	Fundraising	24	46	Cohort & Retention Data	7
22	Cap Table & Ownership	18	47	Financial Statements	6
23	Strategy & Objectives	13	48	Financial Forecast	30
24	Roadmap & Milestones	15	49	Source & Evidence Metadata	13
25	Risks & Mitigation	26			

Appendix D — Theme Library (40)

Family	Themes (key — name, mode)
Classic	meridian — Meridian Navy (dark, default); onyx — Onyx Coral (dark); abyss — Abyss Teal (dark); nocturne — Nocturne Violet (dark); forest — Forest Lime (dark); verdant — Verdant (light); indigo — Indigo SaaS (light); editorial — Editorial Mono (light); terra — Terracotta (light); slate — Slate Amber (light)
Studio	broadsheet — Broadsheet (light); monolith — Monolith (dark); blueprint — Blueprint (dark); aperture — Aperture (dark); ledger — Ledger (light); atrium — Atrium (light); strata — Strata (light); kiosk — Kiosk (light); vellum — Vellum (light); circuit — Circuit (dark)
Volume 2	swiss — Swiss Grid (light); gazette — Gazette (dark); billboard — Billboard (light); bento — Bento (light); boardroom — Boardroom (light); console — Console (dark); aurora — Aurora (light); drafting — Drafting (dark); noir — Noir Luxe (dark); panelled — Split Panel (light)
Infographic	pulse — Data Pulse (dark); flux — Gradient Flow (dark); cobalt — Cobalt (dark); amber — Amber Alert (dark); chrome — Midnight Chrome (dark); atlas — Market Atlas (light); prism — Prismatic (light); sage — Sage (light); crimson — Crimson (light); rosegold — Rose Gold (light)

Appendix E — API Endpoint Inventory

Auth column: **Y** = bearer token required; **N** = public. Limits are per client IP.

Method and path	Auth	Rate limit	Purpose
`GET /health`	N	—	Static liveness response
`GET /uploads/{project}/{file}`	N	—	Static user-uploaded images
`POST /api/v1/auth/register`	N	10/min	Create account, return token
`POST /api/v1/auth/login`	N	10/min	Authenticate, return token
`POST /api/v1/auth/logout`	N	—	No-op acknowledgement
`GET /api/v1/auth/me`	N	—	Placeholder redirecting to `/users/me`
`GET /api/v1/users/me`	Y	—	Current user
`POST /api/v1/projects`	Y	60/h	Create project; optionally start generation
`GET /api/v1/projects`	Y	—	Paginated list (≤100/page), status filter
`GET /api/v1/projects/{id}`	Y	—	Project detail incl. deck content
`PUT /api/v1/projects/{id}`	Y	—	Rename, set theme, save edited deck text
`DELETE /api/v1/projects/{id}`	Y	—	Soft delete
`POST /api/v1/projects/{id}/assets`	Y	—	Upload logo / ≤5 gallery images
`POST /api/v1/projects/{id}/analyze`	Y	20/h	Start (re)generation
`GET /api/v1/projects/{id}/status`	Y	—	Latest job progress
`GET /api/v1/deck-types`	N	—	Deck-type catalogue and intake fields
`GET /api/v1/profiles/schema`	N	—	Parameter catalogue (filter by deck type/stage)
`GET /api/v1/profiles`	Y	—	List the user's profiles
`POST /api/v1/profiles`	Y	—	Create profile
`GET /api/v1/profiles/{id}`	Y	—	Profile with completeness
`PATCH /api/v1/profiles/{id}`	Y	—	Partial merge update
`GET /api/v1/profiles/{id}/readiness`	Y	—	Missing required parameters for a deck type
`DELETE /api/v1/profiles/{id}`	Y	—	Hard delete
`GET /api/v1/decks/{project_id}`	Y	—	Deck content and slide count
`GET /api/v1/presentations/{project_id}/download`	Y	—	Re-render and download PPTX
`GET /api/v1/billing/plan`	Y	—	Current plan and limits

Method and path	Auth	Rate limit	Purpose
`GET /api/v1/billing/plans`	N	—	Public plan listing
`POST /api/v1/billing/checkout`	Y	—	Create Razorpay subscription
`POST /api/v1/billing/verify`	Y	—	Verify payment signature, upgrade
`POST /api/v1/billing/cancel`	Y	—	Cancel at cycle end
`POST /api/v1/webhooks/razorpay`	Signature	—	Subscription lifecycle events
`POST /api/v1/pipeline/generate`	**N**	**none**	**Legacy full pipeline (S-01)**
`GET /docs`, `/redoc`	N	—	Interactive API docs; disabled in production

Appendix F — Environment Variable Catalogue

Variable	Purpose	Production guard
`APP_ENV`, `APP_DEBUG`, `APP_NAME`, `APP_PORT`	Mode and identity	`APP_DEBUG` must be false in production
`APP_SECRET_KEY`	Application secret	Must not be empty / `change_me`
`JWT_SECRET_KEY`, `JWT_ALGORITHM`, `JWT_ACCESS_TOKEN_EXPIRE_MINUTES`, `JWT_REFRESH_TOKEN_EXPIRE_DAYS`	Token signing (refresh setting is unused)	Secret ≥ 32 chars and non-placeholder
`DATABASE_URL`, `DATABASE_POOL_SIZE`, `DATABASE_MAX_OVERFLOW`	Database (pool settings are overridden in code)	Not checked
`SUPABASE_URL`, `SUPABASE_ANON_KEY`	Unused client	—
`REDIS_URL`, `REDIS_CACHE_TTL`, `CELERY_BROKER_URL`, `CELERY_RESULT_BACKEND`	Legacy queue/cache	—
`GEMINI_API_KEY`, `GEMINI_API_KEY_2`, `GEMINI_MODEL`, `GEMINI_MODEL_RESEARCH`, `GEMINI_MAX_TOKENS`, `GEMINI_THINKING_BUDGET`	AI provider	Key and model required
`GROQ_API_KEY`, `ANTHROPIC_*`	Unused	—
`S3_ENDPOINT_URL`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`, `S3_BUCKET_NAME`, `S3_REGION`, `CDN_BASE_URL`	Object storage	**Not checked** (silent local fallback)
`CORS_ORIGINS`, `CORS_ORIGIN_REGEX`	Browser origin allow-list	No `localhost` in production
`RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`, `RAZORPAY_PRO_PLAN_ID`	Billing	Not checked (checkout returns 503 if unset)
`SENTRY_DSN`, `SENTRY_TRACES_SAMPLE_RATE`, `LOG_LEVEL`	Monitoring	Optional
`PLAYWRIGHT_HEADLESS`, `PLAYWRIGHT_BROWSERS_PATH`, `CRAWL_TIMEOUT_MS`, `CRAWL_MAX_PAGES`	Crawler	—
Frontend: `VITE_API_URL`	API base URL, baked at build time	Warns loudly in production console if unset

Appendix G — Test Execution Record

Item	Detail
Date	3 November 2026
Interpreter	Python 3.13 (project container targets 3.11)
Command	`python3 -m pytest -q` over the nine offline modules
Result	**559 passed, 1 skipped, 0 failed**, 4.95 s
Not executed	`test_crawler.py`, `test_pipeline.py` (require Chromium and live network/model access; abort collection offline)
Not run	Frontend (no tests exist), end-to-end, load, penetration and accessibility testing
Caveat	Dependencies were installed ad hoc because the repository provides no lock file

Appendix H — Evidence Index (selected)

Reference	Location
Unauthenticated pipeline endpoint	`ipreneur/backend/app/api/v1/endpoints/pipeline.py:26`
Crawler domain check, no address filter	`ipreneur/backend/app/services/crawler/web_crawler.py:84`
Image fetch with redirects	`ipreneur/backend/app/services/images/image_service.py:62`
Embedded API-key-format strings	`ipreneur/backend/_test_step1.py:5-6` (and `_test_step3/4/5.py`)
Attach profile without owner check	`ipreneur/backend/app/api/v1/endpoints/projects.py:378`, `:88`
Thread-based generation	`ipreneur/backend/app/api/v1/endpoints/projects.py:356`
Error detail leak	`ipreneur/backend/app/api/v1/endpoints/projects.py:400`
Plan limits table; counters unused	`ipreneur/backend/app/api/v1/endpoints/billing.py:58`; `ipreneur/backend/app/models/models.py:306`
Upload allow-list	`ipreneur/backend/app/services/storage/assets.py:21`
Public uploads mount	`ipreneur/backend/app/main.py:183`
Production boot guard	`ipreneur/backend/app/core/config.py:138`
Orphaned-job reconciliation	`ipreneur/backend/app/main.py:48`
Token in `localStorage`	`ipreneur/frontend/src/stores/authStore.ts:81`
Claims auditor wiring	`ipreneur/backend/app/agents/master/master_agent.py:415-427`
Exposed-credential disclosure (log)	`ipreneur/PROJECT_LOG.md` entries #30, #52
Razorpay not configured in production (log)	`ipreneur/PROJECT_LOG.md` entry #59
Frontend deploy via WIF	`.github/workflows/firebase-hosting-deploy.yml`; log entry #61

Appendix I — Glossary

Term	Meaning
ARR / MRR	Annual / Monthly Recurring Revenue
CAC / LTV	Customer Acquisition Cost / Lifetime Value
NRR	Net Revenue Retention
TAM / SAM / SOM	Total / Serviceable / Obtainable market
SSRF	Server-Side Request Forgery — tricking a server into making requests to unintended (often internal) destinations
IDOR	Insecure Direct Object Reference — accessing another tenant's object by its identifier
CSP	Content-Security-Policy browser header
WIF	Workload Identity Federation — short-lived cloud credentials issued to CI without stored keys
DPA	Data-Processing Agreement
DPDP	India's Digital Personal Data Protection Act, 2023
Grounding	Gemini feature that supports answers with live Google Search results and returns source links
R/C/O/A/N	Required / Conditional / Optional / Auto-derived / Not applicable relevance codes
Lakh / Crore	Indian numbering units: 1 lakh = 100,000; 1 crore = 10,000,000

End of report.